

UNIVERSITY OF PLYMOUTH

**SCHOOL OF ENGINEERING, COMPUTING, AND
MATHEMATICS**

**COMP3000
COMPUTING PROJECT
2024/2025**

BAUTH - KEYSTROKE AUTHENTICATION SYSTEM USING MACHINE LEARNING

Stephen Nzekwesi

10736843

BSc (Hons) Cybersecurity

ACKNOWLEDGEMENTS:

I give all glory to God Almighty for the wonderful opportunity to study and successfully complete my program at this great university. My deepest gratitude goes to my family for their unwavering love, encouragement, and support throughout my academic journey.

I extend my sincere appreciation to my project supervisor, Mr. Vivek, for his guidance, support, and invaluable insights during the course of this project. I am also grateful to all my lecturers who have mentored and inspired me throughout my studies.

To my friends and fellow students—thank you for the shared knowledge, constant support, and meaningful collaboration that enriched my learning experience.

Finally, I would like to acknowledge everyone who, in one way or another, contributed to the successful completion of my program. I truly appreciate you all and wish you continued success in all your endeavors.

ABSTRACT:

As cyber threats grow in complexity and frequency, traditional username-password authentication methods are no longer sufficient to protect user accounts and sensitive data. Weak password practices, phishing, and large-scale breaches have exposed the limitations of static credentials. To address these vulnerabilities, organizations are turning to advanced authentication solutions that go beyond conventional methods.

BAuth is an intelligent authentication system that leverages machine learning to analyse and learn from users' unique keystroke dynamics – such as typing speed and rhythm – as a foundation for identity verification. Unlike conventional multi-factor authentication (MFA) approaches, such as one-time passwords or physical tokens that depend on external devices, BAuth operates independently by relying solely on the user's keyboard interactions. This eliminates the need for additional hardware, offering a seamless, cost-effective, and device-free authentication experience. By leveraging typing patterns that are difficult to forge, BAuth enhances security without compromising user experience. The system continuously adapts to users' natural typing variations, ensuring robust and real-time authentication. This approach not only strengthens access control but also offers a scalable, cost-effective solution for organizations aiming to protect digital assets in an increasingly hostile cyber environment.

BAuth demonstrates the potential of keystroke authentication in delivering intelligent, adaptive, and user-friendly security measures, helping organizations stay ahead of evolving cyber threats.

TABLE OF CONTENTS:

ACKNOWLEDGEMENTS	2
ABSTRACT	2
WORD COUNT:.....	5
CODE LINK:.....	5
1. INTRODUCTION:.....	6
2. BACKGROUND, OBJECTIVES AND DELIVERABLES:	6
2.1. Background:	6
2.2. Aims and Objectives:.....	7
2.3. Deliverables:	7
3. ASSUMPTIONS, SCOPE AND LIMITATIONS:.....	9
4. LITERATURE REVIEW:.....	10
5. LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL ISSUES:.....	12
5.1. Legal Issues:.....	12
5.2. Social issues:.....	12
5.3. Ethical Issues:	12
5.4. Professional Issues:	13
6. PROJECT PLANNING:.....	13
6.1. Dataset Aquisition:	13
6.2. Method of Approach:	13
6.3. Technologies:	14
6.3.1. Machine Learning:.....	14
6.3.2. Python:	14
6.3.3. Flask:	15
6.3.4. HTML, CSS, JavaScript, PHP:.....	15
6.3.5. MySQL Database:.....	15
6.4. Best Practices:.....	15
6.4.1. Proper Documentation and Commenting:	15
6.4.2. Backup and Version Control:	16
6.4.3. Code Structure:	16
6.4.4. Security by Design:	16
6.5. Development and Testing Environments:	16
6.6. Risk Assessment and Management:	17
6.6.1. Development Risks and Management:	17
6.6.2. Product Risks and Management:	18
7. SPRINT PLANNING:.....	18
7.1. Sprint Duration and Structure:	18
7.2. Sprints:	18
7.2.1. Sprint 0 – Project Initiation:	18
7.2.2. Sprint 1 & 2 – Project Setup and Requirement Gathering:.....	19

7.2.3. Sprint 3 – Machine Learning Prototype Development:	19
7.2.4. Sprint 4 – API Development and Integration:	19
7.2.5. Sprint 5 & 6 – Frontend Interface Development:	19
7.2.6. Sprint 7 – System Testing and Refinement:	19
7.2.7. Sprint 8 & 9– Finalization and Documentation.....	19
8. PROJECT DESIGN:.....	20
8.1. Requirement Engineering:	20
8.1.1. Functional Requirements:	20
8.1.2. Non-Functional Requirements:	21
8.1.3. Usability Requirements:	21
8.2. User Stories:	22
8.3. UML Diagrams:	22
8.3.1. Class Diagram:	22
8.3.2. Sequence Diagram:	23
8.3.3. Use case Diagram:	23
8.3.4. Activity Diagram:	24
8.3.5. State Diagram:	26
8.4. Wireframe:	27
8.5. Entity Relationship Diagram:	27
9. PROJECT MANAGEMENT:	27
9.1. Scrum:	28
9.2. Version Control:	28
10. PROJECT IMPLEMENTATION:	28
10.1. Feature Engineering:	29
10.2. Model Evaluation:	29
10.3. API Integration:	30
10.4. Web Interface:	30
10.5. Deployment:	30
11. TESTING:	31
11.1. Unit Testing:	31
11.2. Usability Testing:	31
12. END-PROJECT REPORT:	32
12.1. Project Shortcomings:	32
12.2. Project Challenges:	33
12.3. Future Enhancements:	34
12.4. General Evaluation:	35
13. REFLECTIONS:	35
14. CONCLUSION:	35
15. REFERENCE LIST:	36
16. APPENDICES:	37

16.1. User Stories:	37
16.2. Wireframes:	39
16.3. About Dataset:	41
16.4. API Endpoints:	42
16.5. Running the System:	43
16.6. Product Manual:	44

WORD COUNT:

Words - 9417

CODE LINK:

GitHub - <https://github.com/Bukka101/BAuth-keystroke-authentication-system-using-machine-learning-by-stephen-nzekwesi>

1. INTRODUCTION:

As cyber threats continue to escalate in both complexity and scale, traditional username-password authentication mechanisms have proven increasingly insufficient in safeguarding sensitive information. The rise of large-scale password breaches, brute-force attacks, and credential stuffing has exposed the inherent vulnerabilities of static, knowledge-based authentication systems. These developments have necessitated a shift towards more robust and layered security models.

Multi-Factor Authentication (MFA) has emerged as a widely adopted solution, enhancing security by requiring users to provide multiple forms of verification before access is granted. Common MFA implementations include one-time passwords (OTPs), hardware tokens, email verifications, and biometric authentication. While these methods significantly increase security, they often come with trade-offs. Many rely on external devices—such as smartphones, hardware tokens, or specialized biometric scanners—which can introduce logistical, financial, and accessibility challenges, particularly in large-scale or resource-constrained environments. Despite advancements in MFA, behavioural biometrics remains an underexplored yet promising domain. Human behaviour, especially typing patterns, is highly individualized and difficult to replicate. Keystroke dynamics—characteristics such as typing rhythm and speed—offer a unique, continuous, and non-intrusive means of verifying user identity. Unlike traditional MFA approaches, keystroke authentication can operate independently of external devices, thereby reducing both cost and system complexity.

By integrating machine learning, this system can study and adapt to users' natural typing behaviors, accounting for subtle variations while ensuring consistent and accurate verification. This project focuses on implementing keystroke dynamics as a form of behavioral authentication, using machine learning to learn and verify user identity based on their typing patterns.

This report outlines the design, development, testing, and deployment processes involved in building **BAuth**—a scalable, machine learning-based authentication system that can be seamlessly integrated into existing security frameworks to complement traditional methods and strengthen organizational cybersecurity.

2. BACKGROUND, OBJECTIVES AND DELIVERABLES:

2.1. Background:

Traditional authentication systems, primarily based on static credentials such as usernames and passwords, are increasingly vulnerable to security threats including phishing attacks, brute-force attempts, large-scale data breaches, to mention but a few. While Multi-Factor Authentication (MFA) methods—such as one-time passwords (OTPs), physical tokens, and biometric systems—offer enhanced

security, they often depend on external devices or specialized hardware, introducing additional cost, complexity, and usability concerns.

Despite these advancements, the potential of behavioral biometrics—particularly keystroke dynamics—remains largely underutilized. Human typing patterns are inherently unique and difficult to replicate, making them a promising candidate for non-intrusive, continuous authentication. However, effectively capturing, analyzing, and adapting to these behavioral traits requires intelligent systems capable of learning and recognizing individual patterns over time.

This project addresses the need for a cost-effective, seamless, and secure authentication mechanism by developing BAuth, a machine learning–driven system that leverages keystroke dynamics for user verification. The goal is to provide a device-free, behavioral-based authentication layer that enhances existing security frameworks without compromising user convenience. The system is designed specifically for organizations who need a secure, easy-to-manage method of user authentication, offering a continuous, non-intrusive layer of security that doesn't compromise user convenience.

2.2. Aims and Objectives:

- To design and develop an authentication system that utilizes keystroke dynamics for user verification.
- To apply machine learning algorithms capable of learning and adapting to individual typing patterns over time.
- To evaluate the effectiveness of keystroke-based authentication in identifying legitimate users and detecting imposters.
- To implement a device-independent authentication method that eliminates the need for external hardware, such as OTP devices or biometric scanners.
- To create a scalable and cost-effective solution that can be integrated with existing authentication frameworks.
- To improve user experience by offering a seamless and non-intrusive form of multi-factor authentication.
- To analyze the performance of the system in terms of accuracy, usability, and resistance to forgery or impersonation.

2.3. Deliverables:

1. Research Documentation
 - A comprehensive literature review on keystroke dynamics and behavioral biometrics.
2. System Design Documentation

- Architecture and design specifications of the BAuth system.
 - UML diagrams, use-case scenarios, and data collection strategies.
3. The Bauth system, which consist of the following:
 - a) RESTful API for model training and classification
 - An API endpoint capable of receiving training data and returning a trained model. Supports multiple model types, including Random Forest and Support Vector Machine (SVM).
 - An API endpoint capable of receiving pre-captured keystroke data in JSON format. The API processes incoming data and returns classification results based on the active trained model.
 - b) Machine Learning Model
 - Implementation of a trained machine learning model capable of learning and distinguishing individual typing patterns.
 - Model performance evaluation metrics (accuracy, false positive/negative rates, etc.).
 - c) Authentication System Prototype
 - A working prototype system integrated with a login interface for practical demonstration.
 - Real-time verification based on captured keystroke dynamics.
 - d) Model Activation and Verification System
 - Functionality to deploy and switch between trained models for live authentication.
 - Ensures that verification requests are processed using the currently active model for accurate user classification.
 - e) Database system and user data
 - A relational database system storing data of users, and data required for the entire system performance.
 - Can be manipulated to alter data.
 4. Testing and Evaluation Report
 - Results from system testing, including performance benchmarks, usability feedback, and potential security assessments.
 5. Final Project Report
 - A detailed written report documenting all stages of the project: background, objectives, methodology, implementation, results, challenges, and conclusions.
 6. Presentation/Defense Materials
 - A visual presentation (e.g., PowerPoint or slides) summarizing the project scope, development, and key findings for academic review or defense.
 - A poster presentation of the project for showcase

3. ASSUMPTIONS, SCOPE AND LIMITATIONS:

- The BAuth system does not include the functionality for capturing keystroke data. Instead, it is designed to operate exclusively on keystroke data that has already been collected and preprocessed by external means. Keystroke capture can be achieved using a variety of techniques, such as deploying and configuring keyloggers or other monitoring tools that record users' typing patterns during login sessions. However, these processes fall outside the scope of BAuth and are expected to be facilitated by separate systems or software solutions.
- BAuth requires that keystroke data be properly formatted and structured to suit its classification requirements. The system assumes that, at the time of authentication, this data has already been collected, cleaned, and formatted in a specific structure—typically in JSON format. This JSON-formatted input is then passed to BAuth's machine learning engine for real-time classification and user verification.
The expected data structure and formatting specifications are discussed in the appendix 16.3.
- For effective model training, the BAuth system requires a dataset of user's keystroke data that has been formatted according to a predefined structure compatible with the underlying program logic. Datasets that do not adhere to this required format may result in processing errors or failed model training attempts. It is therefore essential that data used for training is properly structured to ensure compatibility and optimal performance.
The specific data format requirements, including field arrangement and expected data types, are detailed in the appendix section of this report 16.3.
- BAuth focuses solely on keystroke dynamics for behavioral authentication. It does not integrate with other behavioral or biometric modalities (e.g., mouse movements, facial recognition), which could further enhance security and reliability.
Although it incorporates username-password, it does not focus on the security and robustness of this functionality. This functionality is only there to serve as a primary authentication method because keystroke is originally intended as a secondary authentication method.
- While typing patterns are difficult to forge, they are not entirely immune to imitation. An attacker with access to previous keystroke data may attempt to replicate user behavior, potentially reducing system effectiveness.

- Effective model performance is dependent on the quality and volume of training data. Small or unbalanced datasets may lead to poor generalization and reduced accuracy when verifying users.
- The BAuth system implements fixed-text keystroke authentication, as opposed to free-text authentication. During the login process, users are required to type a predefined, fixed text. Their typing behavior is then monitored and recorded by the external capturing system. This captured pattern is subsequently passed to the system's API, where it undergoes machine learning classification to verify the user's identity.

4. LITERATURE REVIEW:

The concept of keystroke authentication goes as far back as 1980, when four researchers Gaines et al introduced the idea of identifying users based on their typing patterns. They experimented with six typists typing three separate text passages over time, capturing and recording their keypress timing and successive digraphs. The recorded times were compared, and the result showing that each digraph generally matches the respective pair with a slight variance. It was concluded that it is possible to identify an individual based on the way they type.

In recent years, keystroke dynamics has gained significant attention as a viable biometric for behavioral authentication. A notable study by AbdelRaouf et al. (2023), titled *"Efficient Convolutional Neural Network-Based Keystroke Dynamics for Boosting User Authentication"*, explores the application of deep learning—specifically Convolutional Neural Networks (CNNs)—to enhance authentication accuracy through analysis of typing patterns. The researchers propose a CNN-based framework capable of learning and identifying unique timing characteristics and typing rhythms of users to distinguish legitimate individuals from impostors. Their results demonstrate that CNNs can effectively model keystroke features and achieve high classification accuracy without imposing usability constraints on users. While BAuth does not currently employ CNNs, the findings underscore the potential of machine learning in strengthening identity verification based on keystroke behavior.

The study by Wang, X. and Hou, D. (2024), titled *"Enhancing Keystroke Dynamics Authentication with Ensemble Learning and Data Resampling Techniques"*, explores how combining ensemble learning models (such as Random Forests and Gradient Boosting) with data resampling methods significantly improves the accuracy of keystroke dynamics-based authentication systems. The authors address key challenges like class imbalance and variability in typing behavior.

Wang et al. (2022) introduced a novel user authentication method that integrates keystroke dynamics and mouse dynamics, aiming to enhance authentication accuracy across various user interaction scenarios. Their approach, termed SIURUA (Scene-Irrelated and User-Related feature-based User Authentication), focuses on extracting features that are less influenced by specific scene contexts, thereby improving the robustness of user authentication systems in hybrid environments. By combining scene-irrelated features with user-related behavioral patterns, the method addresses the variability in user behavior across different applications and tasks. Experimental results demonstrated that the SIURUA method achieved an authentication accuracy of 84%, highlighting its potential effectiveness in real-world applications.

Wang et al. (2020), in their publication "*User Authentication Method Based on MKL for Keystroke and Mouse Behavioral Feature Fusion*", proposed a user authentication method that integrates keystroke and mouse dynamics using Multiple Kernel Learning (MKL) to enhance authentication accuracy across diverse user interaction scenarios. The study introduces the concept of scene-irrelated features, aiming to extract behavioral characteristics that remain consistent regardless of the specific context or application in use. By focusing on these invariant features, the method seeks to improve the robustness and generalizability of authentication systems in hybrid environments. Experimental results demonstrated that the proposed approach effectively distinguishes legitimate users from impostors, highlighting its potential for real-world applications.

Elliot et al. (2019) in their publication titled "*A Comparison of Machine Learning Algorithms in Keystroke Dynamics*", experimented with four machine learning algorithms – Decision Trees, Random Forest, Support Vector Machines, and Neural Networks – to determine which is most effective on accuracy. Their result proves that Random Forest is most effective in classifying users to their typing patterns, making their findings especially relevant to systems like BAuth, which also looks to leverage machine learning for keystroke authentication.

While not all the above reviewed literatures are directly relevant to the proposed system, they provide understanding and influence the implementation of BAuth. Ideas were drawn upon them in understanding and implementing the desired model for BAuth keystroke authentication based on the available dataset.

Dataset sourcing was one of the challenges encountered during this project, which will be discussed later down this report. Considerations were based on the already acquired dataset, for the implementation of this system. At the end of the research and reviews, it was understood that Random Forest performed best when the dataset is of similar size to the size of the acquired dataset.

5. LEGAL, SOCIAL, ETHICAL AND PROFESSIONAL ISSUES:

The implementation of BAuth raises several critical legal, social, ethical, and professional concerns. While the system provides an innovative and cost-effective alternative to traditional multi-factor authentication methods, it also requires careful consideration of how it aligns with legal requirements, impacts society, upholds ethical standards, and conforms to professional practice.

5.1. Legal Issues:

One of the most pressing legal concerns involves data protection and privacy regulations. Keystroke data collection and usage are subject to data protection laws, especially as keystroke dynamics are considered a form of biometric data, which is classified as sensitive personal data under legislation such as the General Data Protection Regulation (GDPR) in the EU, or the Data Protection Act (DPA) in the UK. Even though BAuth does not collect data itself, organisations using it must ensure the captured data is lawfully collected with user consent, stored securely and used only for the stated purpose.

Moreover, legal responsibility must be established in the event of system failure or misuse. If an individual is wrongly denied access or if a breach occurs due to false acceptance, questions of liability and compensation may arise. Organisations must therefore ensure their systems are rigorously tested and that appropriate fallback mechanisms are in place.

5.2. Social issues:

From a social perspective, user acceptance of behavioural authentication methods like BAuth can be a barrier to adoption. Users may feel uncomfortable with the idea of being "monitored" through their typing behaviour, especially if they lack understanding of how the data is used (Van der Ploeg, 2005).

Additionally, concerns around digital accessibility must be considered. Users with motor impairments, neurological conditions, or temporary injuries may exhibit inconsistent typing behaviours, leading to frequent authentication failures and digital exclusion. To maintain inclusivity, developers must consider flexible thresholds or alternative verification paths for such users (AlZain and Pardede, 2013).

5.3. Ethical Issues:

Ethically, the principle of informed consent is fundamental. Even though BAuth does not collect data itself, systems built on it must ensure users are aware of and agree to the use of their typing patterns for authentication. Ethical use demands full disclosure on how data is processed, stored, and protected (Floridi et al., 2018).

Another ethical concern is data security and anonymisation. Although typing patterns are not as sensitive as passwords, they are still personal behavioural data and must be protected from breaches or misuse.

Furthermore, algorithmic transparency is a concern. If a user's access is denied due to a machine learning decision, they should have a clear explanation or recourse.

Black-box models—like Random Forest—pose a risk of non-accountability, where neither the system nor its administrators can justify decisions (Burrell, 2016). This challenges ethical principles of fairness and transparency.

5.4. Professional Issues:

Professionally, integrators of BAuth must adhere to the ethical codes and standards of practice such as the ACM Code of Ethics and the BCS Code of Conduct. These codes emphasise responsibilities including the public good, honesty, competence, and respect for privacy.

Additionally, maintaining high software quality standards is critical. The system must undergo rigorous testing for accuracy and robustness before integration. Inadequate testing can lead to security loopholes or usability issues, undermining the system's purpose and reflecting poorly on the developers' professionalism.

Continuous maintenance and model retraining are also necessary, especially since human behaviour evolves over time. Failure to update the model could reduce accuracy and compromise security (Jain et al., 2016). It is a professional obligation to ensure that machine learning models used in BAuth remain current and adaptable.

6. PROJECT PLANNING:

6.1. Dataset Aquisition:

Sourcing a suitable dataset for the implementation of this project presented a significant challenge due to the limited availability of publicly accessible keystroke dynamics datasets. The field of keystroke-based authentication remains relatively underexplored, which may contribute to the scarcity of standardized datasets tailored for authentication purposes.

Initially, the project aimed to implement a free-text keystroke authentication model. However, the unavailability of appropriate and well-structured datasets led to a strategic pivot toward fixed-text authentication, which proved more feasible under the circumstances. Despite this shift, the availability of high-quality datasets remained limited.

Comprehensive searches were conducted using prominent data repositories and search engines, including Kaggle, Google Dataset Search, EU Open Data Portal, and Data.gov (USA). Among these, Kaggle was the only platform that provided a well-formatted and context-appropriate dataset suitable for the project's requirements. This dataset was subsequently adopted as the foundational benchmark for the development, training, and evaluation of the BAuth system.

6.2. Method of Approach:

The development of the BAuth system was guided by the Agile software development methodology, a strategy that emphasizes flexibility, iterative progress, and continuous feedback. Agile was preferred over traditional methodologies due to

the dynamic nature of the project and the need for frequent adjustments in response to findings from testing, user interface considerations, and model performance.

Agile development allowed the project to be broken down into incremental phases or sprints, each focusing on specific deliverables such as building the machine learning training module, integrating the keystroke classification API, and designing the frontend user interface. These sprints facilitated continuous delivery of functional components, enabling early detection of issues and quick adaptation to evolving requirements. For instance, the pivot from freehand keystroke input to fixed-text input was efficiently managed within an Agile framework without disrupting overall progress.

Regular iterations provided opportunities to test, evaluate, and refine the system's performance, particularly in model training and prediction accuracy. Additionally, Agile enabled tight collaboration between the backend development of the machine learning components and the frontend implementation of the GUI. Feedback loops within sprints supported usability enhancements and API refinements that contributed to a more intuitive user experience.

Ultimately, the Agile methodology contributed to building a scalable, modular, and adaptable authentication system. Its flexibility ensured that BAuth remained aligned with its core goals—accuracy, simplicity, and device-free verification—while maintaining responsiveness to technical challenges and new insights throughout the development lifecycle.

6.3. Technologies:

The BAuth authentication system was developed using a carefully selected combination of technologies, designed to meet the demands of flexibility, performance, and integration in behavioural biometric authentication. These technologies include machine learning frameworks, programming languages, web development tools, and a relational database management system — each playing a vital role in the system's end-to-end functionality.

6.3.1. Machine Learning:

At the heart of BAuth lies the implementation of Machine Learning, specifically for user identity verification through keystroke dynamics. Algorithms such as Support Vector Machines (SVM) and Random Forests were employed due to their strong performance in classification tasks, especially with time-series and behavioural data. These models were chosen for their ability to generalize well across users, maintain high prediction accuracy, and handle relatively small datasets efficiently.

6.3.2. Python:

Python was the primary language used for implementing the machine learning components. Python was chosen because of its rich ecosystem of libraries such as

scikit-learn, pandas, and NumPy, which streamline data preprocessing, model training, and evaluation. Python's readability and strong community support also contributed to faster development and troubleshooting.

6.3.3. Flask:

A key architectural feature of BAuth is that both model training and prediction are handled via RESTful API endpoints, built using Flask, a lightweight and powerful micro-framework. This approach decouples the machine learning logic from the frontend, making the system more modular, scalable, and easy to maintain.

6.3.4. HTML, CSS, JavaScript, PHP:

On the frontend, technologies such as HTML, CSS, and JavaScript were used to build the user interface. These standard web technologies ensured a responsive, accessible, and interactive experience for end-users. PHP was integrated primarily for handling server-side logic where necessary, and for managing form submissions and database interactions within a traditional web environment.

6.3.5. MySQL Database:

The application's data layer was supported by a MySQL database, a widely adopted relational database management system. MySQL was chosen due to its reliability, structured query language support, and ease of integration with both PHP and Python. It facilitated the secure storage of user details, training data, model metadata, and model performance data.

The combination of these technologies provided a robust, modular, and scalable architecture for BAuth. Each technology was selected not only based on its capabilities but also its compatibility with others in the stack. The overall design supports maintainability, ease of integration into existing systems, and future extensibility, ensuring that BAuth can evolve to meet emerging authentication needs.

6.4. Best Practices:

6.4.1. Proper Documentation and Commenting:

Through the development, documentation and code commenting played a vital role in ensuring maintainability and collaboration. Clear and comprehensive documentation was maintained for both the system's architecture and its individual components, including APIs, machine learning models, and data workflows. This allows new developers to quickly understand the system, its dependencies, and how different modules interact. Code commenting within the codebase was equally important, particularly in complex areas such as the machine learning model training and classification processes. Well-commented code explains the logic behind key decisions, making it easier for developers to troubleshoot, extend, or improve upon

the system in the future. It also helps in understanding data preprocessing steps, model training, and prediction handling, which are integral to BAuth's functionality.

6.4.2. Backup and Version Control:

Another best practice was backup and version control. Using a version control system like Git allowed management of changes to the codebase, efficient collaboration, and tracking the history of all modifications. Version control also mitigates the risk of losing code due to unforeseen events and enables developers to revert to previous, stable versions if needed. Regularly backing up both the codebase and critical data (such as trained models) ensured that the project remained secure at all times.

6.4.3. Code Structure:

Code structure is another best practice that greatly affects the maintainability and scalability of the system. Using a framework such as the Model-View-Controller (MVC) architecture helped organize the code in a clear and structured manner. Structuring the code in this way promotes modularity, making it easier to make changes or upgrades without affecting the whole system. It also supports separation of concerns, which ensures that the user interface, business logic, and data handling are independently managed.

6.4.4. Security by Design:

Security considerations were integrated into every stage of development lifecycle. Since BAuth involves sensitive user data, including keystroke dynamics, incorporating secure coding practices and encrypting sensitive data from the outset is essential for protecting users and complying with data protection regulations (such as GDPR).

6.5. Development and Testing Environments:

The development and testing of the BAuth system were carried out across two primary environments, reflecting the system's dual architecture: the machine learning component and the web-based interface.

For the machine learning aspect, Google Colaboratory (Colab) served as the primary development and testing environment. Colab provides a cloud-based platform with support for Python and relevant machine learning libraries, allowing for efficient experimentation and model evaluation. It enabled the training and validation of classification models using performance metrics such as accuracy score to assess effectiveness.

In parallel, the web interface of the BAuth system was developed using Visual Studio Code (VS Code), a widely adopted integrated development environment (IDE) known for its versatility and robust support for web technologies. The system's frontend components—built using HTML, CSS, JavaScript, and PHP—were

continuously tested in modern web browsers to ensure compatibility, responsiveness, and functionality.

This dual-environment approach ensured that both core components of the system were developed and validated using tools best suited to their respective requirements, promoting efficiency, maintainability, and scalability.

6.6. Risk Assessment and Management:

The risk assessment for the BAuth project is categorized into two broad areas: risks associated with the development process and those related to the deployed product.

6.6.1. Development Risks and Management:

1. Time Constraints and Conflicting Commitments:

Description: As an international student managing academics, work obligations, and personal responsibilities, time posed a significant challenge during the course of this project. Balancing multiple module assignments alongside this dissertation required efficient time management.

Risk Factor: High

Likelihood: Common

Mitigation: A detailed project schedule and task breakdown were implemented using Jira to follow the Agile development model. A personal productivity system was adopted, including self-imposed accountability for time lost. Regular bi-weekly meetings with the project supervisor also ensured continuous progress and helped maintain direction and focus.

2. Potential Failure of Development Laptop:

Description: The primary development system—a personal laptop—was relatively old and could potentially malfunction under the high computational demands of machine learning tasks.

Risk Factor: High

Likelihood: Low

Mitigation: Preventive optimization measures were applied, including the removal of redundant files and applications. A contingency fund was reserved for emergency repairs or replacement. Additionally, arrangements were made to borrow a backup laptop from a friend if necessary.

3. Risk of Data Loss:

Description: Unintentional data loss during development posed a significant threat to progress, especially given the strict project deadlines.

Risk Factor: High

Likelihood: Common

Mitigation: Version control was rigorously implemented using GitHub, with frequent commits ensuring data was preserved at all stages. This also doubled as a backup strategy in the event of device failure.

6.6.2. Product Risks and Management:

Description: Once deployed, BAuth—like any web-facing application—is exposed to a variety of potential security risks. These include SQL injection, man-in-the-middle (MITM) attacks, replay attacks, data leakage, and ransomware attacks targeting the underlying infrastructure, to mention only a few.

Risk Factor: Variable

Likelihood: Variable (depending on the specific threat)

Mitigation: The system was developed using the Security by Design principle to address threats at the architectural level by avoiding common vulnerabilities and enforcing secure development practices. While the development phase focused on embedding robust security, it is also recommended that comprehensive production-level security measures such as secure API gateways, regular patching, encryption standards, and threat detection systems be implemented in future integration or deployment phases.

7. SPRINT PLANNING:

The development of BAuth followed an Agile methodology, with the project timeline divided into iterative sprints that allowed for progressive refinement, continuous feedback, and adaptability. Sprint planning was essential in organizing and prioritizing tasks while ensuring that development remained focused and manageable within the constraints of time and resources.

Sprint planning ensured that each development phase had clearly defined goals, which helped track progress and stay aligned with the project's broader timeline. The use of tools like Jira supported backlog tracking, task prioritization, and time allocation throughout the process.

7.1. Sprint Duration and Structure:

Each sprint was scheduled to last two weeks, aligning with academic deadlines and personal commitments. This duration was considered optimal for balancing development progress with regular academic responsibilities. Sprint planning sessions were conducted at the start of each sprint to identify key deliverables, assign tasks, and estimate efforts required.

7.2. Sprints:

7.2.1. Sprint 0 – Project Initiation:

- Identified and finalized project topic.
- Defined project vision and high-level requirements.
- Conducted initial literature review to assess the feasibility and relevance of the project.
- Drafted initial risk assessment and management plan.

7.2.2. Sprint 1 & 2 – Project Setup and Requirement Gathering:

- Defined system goals and objectives.
- Identified core functionalities.
- Set up project repository and version control via GitHub.
- Conducted research and sourced datasets.
- Drafted initial system architecture.

7.2.3. Sprint 3 – Machine Learning Prototype Development:

- Preprocessed keystroke dataset.
- Performed manual feature engineering.
- Built and tested basic machine learning classifiers.
- Evaluated initial model performances.
- Defined API requirements for integration.

7.2.4. Sprint 4 – API Development and Integration:

- Developed Flask-based API for model training and prediction.
- Created JSON-based input schema for keystroke data.
- Integrated model loading and classification into API endpoints.
- Tested API locally using Postman and test scripts.
- Implemented logging and error handling.

7.2.5. Sprint 5 & 6 – Frontend Interface Development:

- Developed GUI using HTML, CSS, JavaScript, PHP, and MySQL.
- Integrated frontend with Flask API.
- Implemented input validation and user feedback.
- Designed a model training panel for uploading dataset and triggering training.

7.2.6. Sprint 7 – System Testing and Refinement:

- Conducted functional and integration testing.
- Validated system response against different datasets.
- Addressed edge cases and improved error messages.
- Documented training dataset format requirements.
- Applied final security and performance optimizations.

7.2.7. Sprint 8 & 9– Finalization and Documentation

- Final system walkthrough and user acceptance testing.
- Prepared technical documentation and user guide.
- Compiled project report.
- Reviewed and organized project files and commits.
- Prepared presentation materials and poster.

8. PROJECT DESIGN:

8.1. Requirement Engineering:

Requirement engineering played an essential role in defining the capabilities, constraints, and expectations from the system. Through an iterative process involving requirement elicitation, specification, validation, and management, both functional and non-functional requirements were derived to ensure that the system meets its intended purpose and aligns with the goals of secure and efficient authentication.

Stakeholders considered during this phase included the system administrator (as the sole user of the application for it is intended to simulate an admin system for management of users), the developer, and future potential users whose data would be analyzed. Agile methodologies, particularly sprint planning sessions, were employed to revisit and refine requirements throughout the development cycle.

8.1.1. Functional Requirements:

For the BAuth system, the following are the core functional requirements:

1. User Management
 - View, edit, and delete users from the system database.
 - Assign specific users to use keystroke-based authentication.
2. Model Management
 - Upload datasets and initiate model training (Random Forest or SVM).
 - Specify the model name, dataset, and model type.
 - Download and store trained model files (trained model and scaler).
 - View, edit, and delete trained models.
 - View model performance metrics including true/false positives and negatives.
 - Activate only one model at a time for classification purposes.
3. Authentication Configuration
 - Configure authentication type for users. Default as username-password. Optionally, users can be assigned to use keystroke-based authentication.
4. Login Testing Interface
 - Simulate user login page, by entering username, password, and optional keystroke data.
 - Verify login credentials and invoke API-based classification if user is assigned for keystroke authentication.
 - Display predicted result.
 - Allow admin to manually label the classification outcome to update model performance.
5. API Integration

- Perform model training and prediction via API endpoints using JSON inputs.

8.1.2. Non-Functional Requirements:

Non-functional requirements describe the qualities and constraints of the system, focusing on how the system performs rather than what it does.

1. Performance
 - The API should respond to classification requests within seconds.
2. Reliability
 - The system should handle malformed or missing JSON data gracefully.
 - Model performance tracking should ensure accurate metric logging.
3. Security
 - Input validation and secure communication between frontend and API.
 - User data and trained models should be protected from unauthorized access.
4. Scalability
 - The system architecture should allow integration of new models or authentication methods (such as mouse movement authentication) in the future.
5. Maintainability
 - Modular code design with consistent documentation and version control practices (e.g., GitHub).

8.1.3. Usability Requirements:

Since BAuth is intended solely for administrator use, usability is streamlined for technical efficiency.

1. Admin Dashboard
 - Simple, intuitive graphical interface to navigate menus for Users, Models, Authentication, and Testing.
 - Visual feedback for each action (e.g., model training confirmation, authentication result).
2. Training and Testing Panels
 - Clear form fields with labels and prompts for dataset selection and model configuration.
 - Easy interpretation of classification results and performance metrics through visual buttons and charts.
3. Error Handling and Feedback
 - Prompt and informative error messages when input formats are invalid or system failures occur.
 - Notification upon successful actions like training completion.
4. Accessibility
 - Designed for ease of use in standard web browsers.
 - Minimal reliance on external dependencies for ease of deployment.

8.2. User Stories:

Descriptive user stories for the system are included in the appendix 16.1., outlining the various actions that could be performed by the administrator user through the system.

8.3. UML Diagrams:

8.3.1. Class Diagram:

The architecture of the system followed a modular and object-oriented design, separating core responsibilities across logically independent classes. Below diagram (fig8.3.1) is a detailed summary of the core classes, their attributes, methods, and relationships.

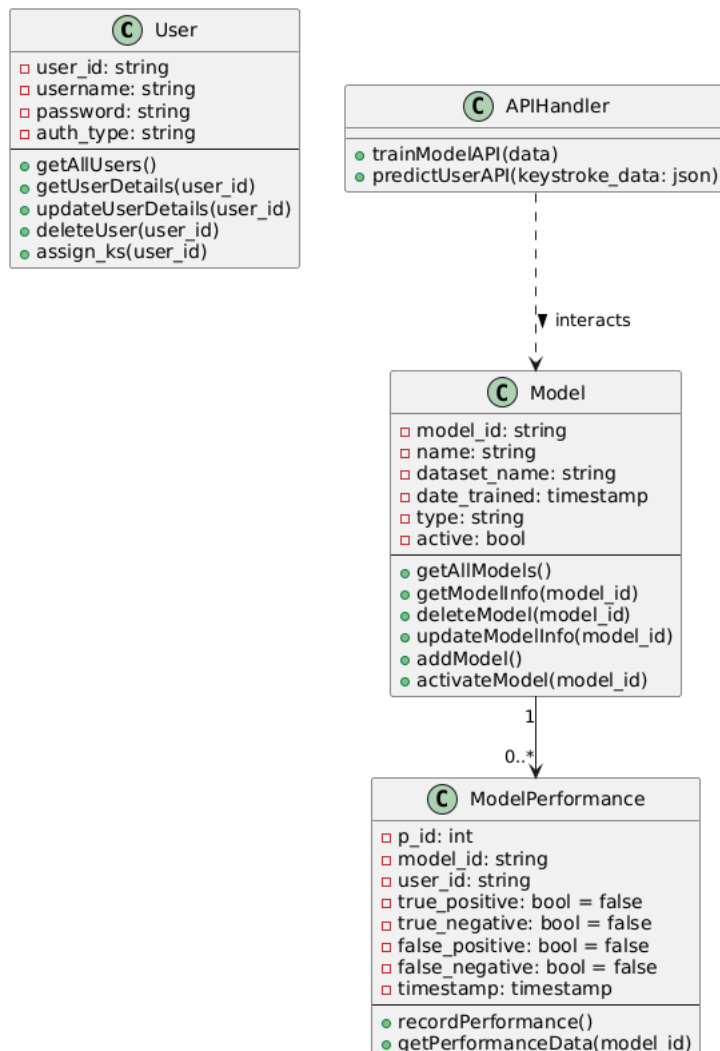


Fig8.3.1- Class diagram

8.3.2. Sequence Diagram:

The sequence diagram (*fig 8.3.2 below*) illustrates the dynamic interaction among the major components of the system during the login testing process. It showcases how the Admin interacts with the system through the web interface, and how API calls facilitate communication between different system components, particularly for prediction and performance tracking.

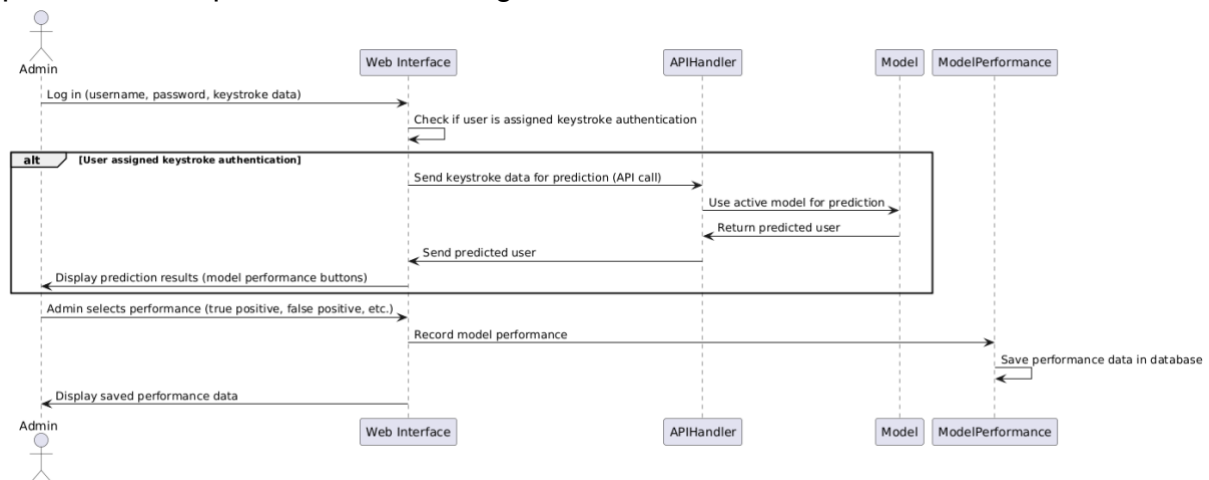


Fig 8.3.2 – Sequence diagram

8.3.3. Use case Diagram:

The use case diagram outlines the key functionalities available for the admin user, focusing on both user and model management, along with authentication-related tasks.

The admin is the primary user who interacts with the system and performs key tasks related to user management, model handling, and authentication settings. The diagram (*fig 8.3.3*) below reflects the admin's interactions with the system.

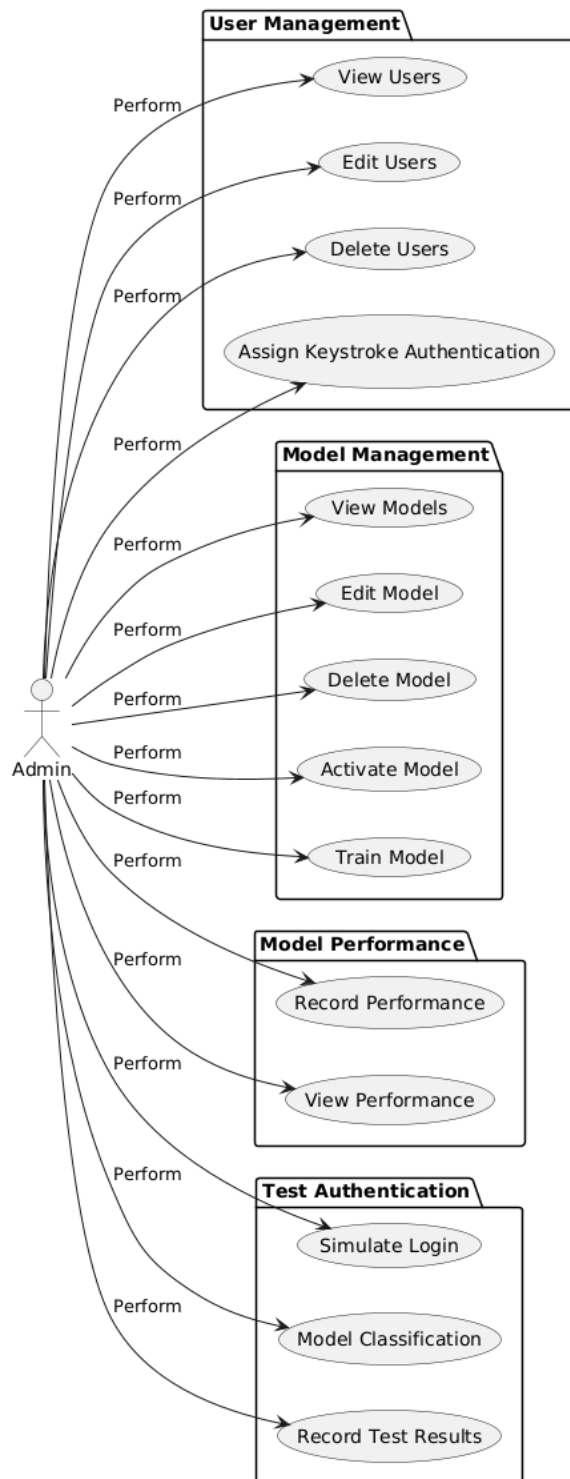


Fig 8.3.3 – Use case diagram

8.3.4. Activity Diagram:

This activity diagram (*fig 8.3.4 below*) illustrates the high-level flow of operations performed by the Admin in the system. It captures the main decision points, parallel processes, and system actions.

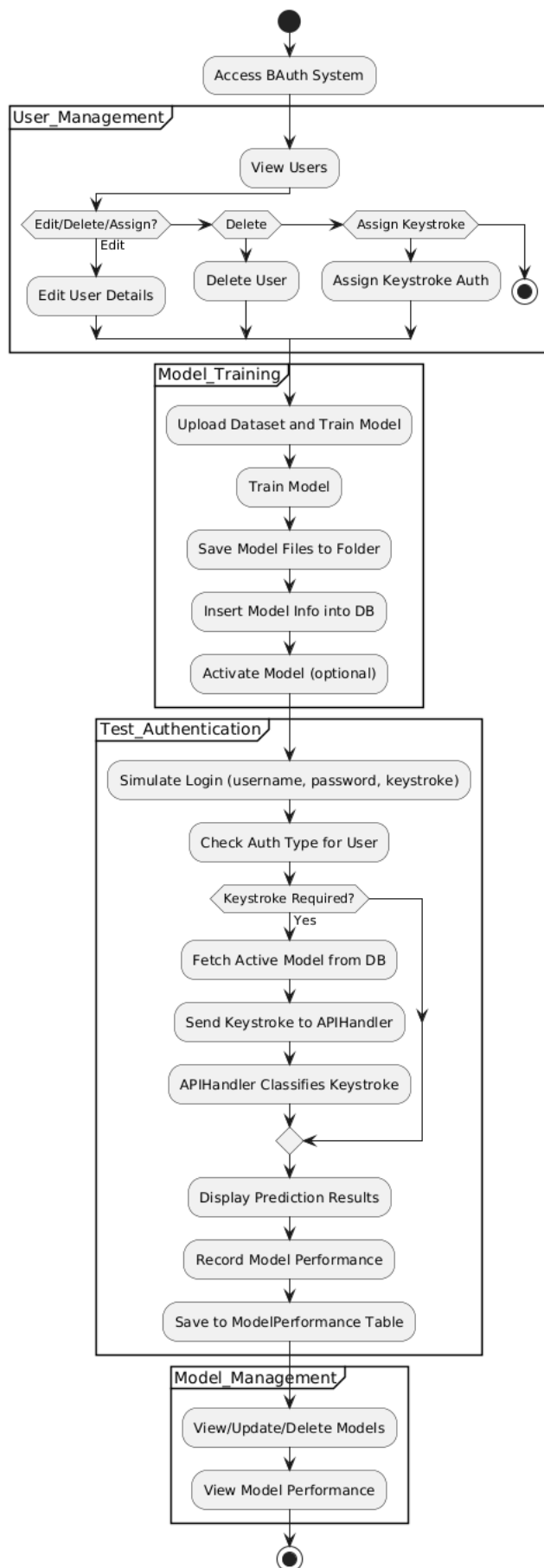


Fig 8.3.4 – Activity diagram

8.3.5. State Diagram:

The system follows a clear sequence of states from the moment the admin initiates a login test to the point where the model performance is recorded. Below diagram (fig 8.3.5) is a visual of these states.

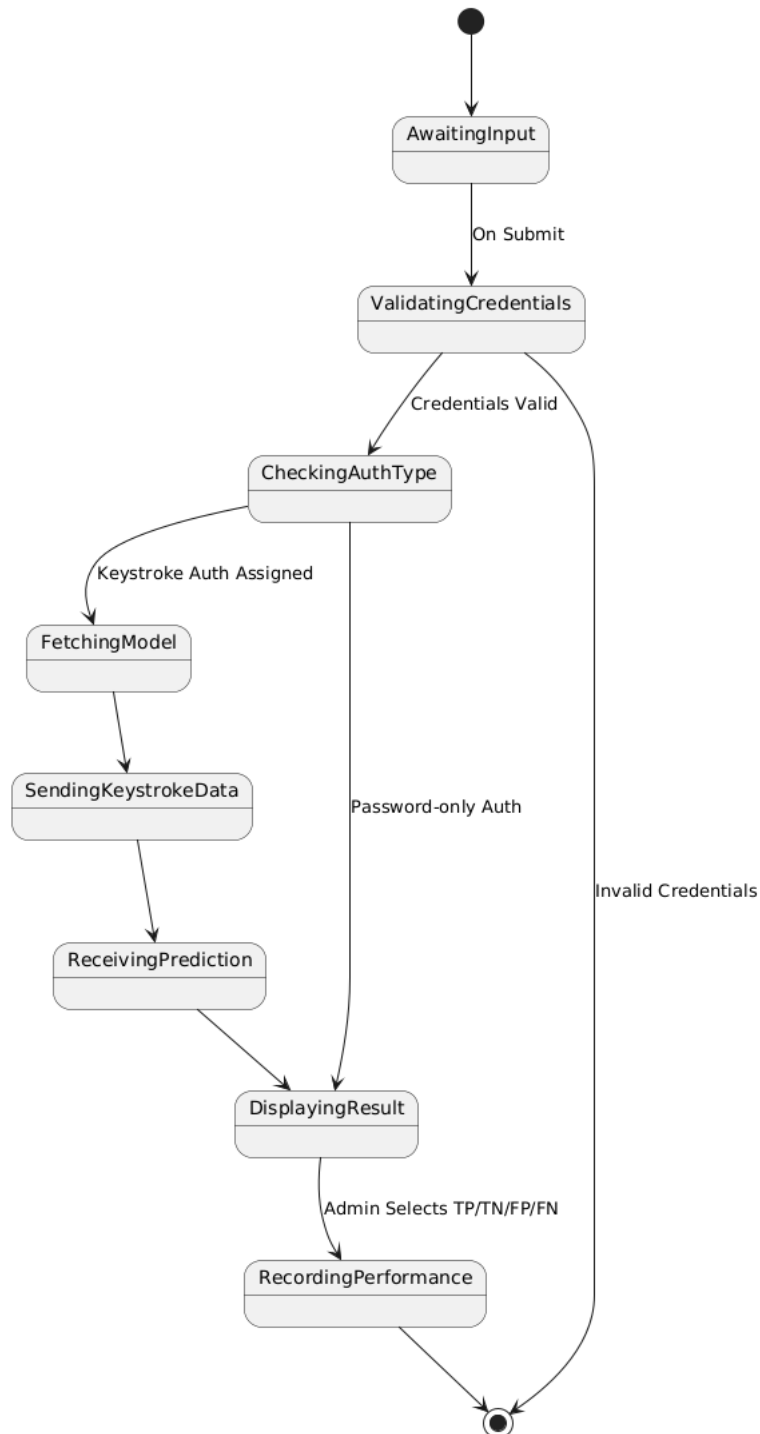


Fig 8.3.5 – State diagram

8.4. Wireframe:

The BAuth system wireframe is structured around simplicity, usability, and functional clarity for admin users. It consists of a top-level navigation menu and dynamic panels based on the selected feature. This is detailed in the appendix 16.2.

8.5. Entity Relationship Diagram:

The ER diagram for BAuth system illustrates three core entities: User, Model, and ModelPerformance. The User entity holds user credentials and authentication type, but has no direct relationship with other entities. The Model entity stores information about each trained machine learning model, including its type, dataset, and activation status. A one-to-many relationship exists between Model and ModelPerformance, where each model can have multiple associated performance records. The ModelPerformance entity logs classification outcomes (true positive, false positive, etc.) for analysis, linked only to the corresponding model, ensuring users remain decoupled from model evaluation data. These are represented in *fig 8.5* below.

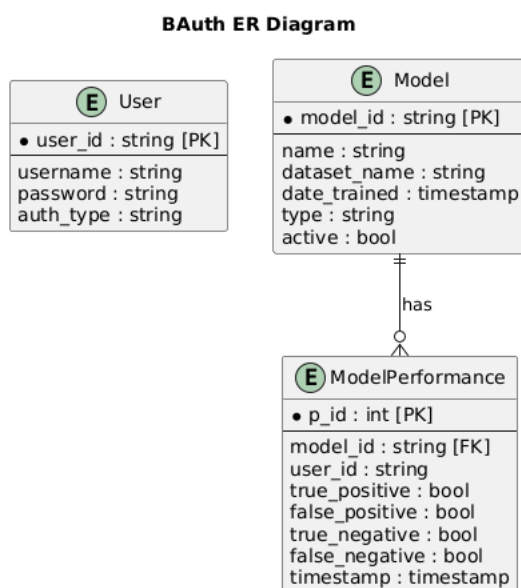


Fig 8.5 - ERD

9. PROJECT MANAGEMENT:

The BAuth system was managed using an Agile-inspired approach, structured around iterative sprints that promoted flexibility, regular progress tracking, and timely delivery. Project planning began with Sprint 0, where the project vision, objectives, and initial risks were identified. Tools like Jira were employed to manage the product backlog, allocate tasks, and maintain workflow clarity. Each sprint had well-defined goals and deliverables, ensuring that both the machine learning and web interface components progressed in parallel. Regular supervisor meetings and self-imposed

accountability measures helped to overcome time-related constraints and maintain consistent development momentum throughout the project lifecycle.

9.1. Scrum:

The Scrum framework guided the development of the system by promoting adaptive planning, incremental progress, and regular feedback. The project was divided into two-week sprints, each beginning with a sprint planning session to define goals and prioritize tasks. A product backlog was maintained in Jira, enabling effective task tracking and refinement throughout the project. Although a formal Scrum team structure was not adopted, core Scrum roles were reflected in practice — with the developer acting as both Scrum team member and Product Owner, while the project supervisor informally assumed the role of Scrum Master by offering bi-weekly guidance and performance reviews. Sprint reviews and retrospectives occurred informally, focusing on what had been achieved and identifying areas for improvement. This adaptation of Scrum ensured that development remained focused, iterative, and responsive to challenges.

9.2. Version Control:

Version control was a critical component in the development of the system, ensuring that changes made to the codebase were tracked, organized, and recoverable. Git was used as the primary version control system, with GitHub serving as the remote repository for collaboration, backup, and deployment management.

The project repository was initialized early in the development phase and structured to reflect both the machine learning and web interface components. Frequent commits were made at each significant development milestone, allowing easy navigation between feature versions, identification of bugs, and rollback to previous states when necessary.

This structured version control approach not only facilitated better organization and reliability throughout the development process but also supported collaboration and review by supervisors. In the event of system failure or data loss, GitHub served as a reliable backup, ensuring the continuity of the project.

10. PROJECT IMPLEMENTATION:

The implementation of the system followed a structured approach, beginning with data processing and feature engineering, where key aspects of users' keystroke patterns were processed to enhance model accuracy. Machine learning models were then trained and evaluated using performance metrics such as accuracy score, ensuring their effectiveness for keystroke authentication. An essential part of the system was the API integration, where a Flask-based backend served as the interface between the trained models and the web frontend, enabling model training and prediction. The frontend was developed to simulate a user management system for administrators to manage users, view models, and perform authentication tests,

offering an intuitive user experience. The implementation was carried out in phases, starting with data preprocessing and model development, followed by backend and frontend integration, and concluding with system deployment in a local environment for testing. Each of these stages are discussed further in subsequent sections to provide a detailed overview of the system's development and functionalities.

10.1. Feature Engineering:

Feature Engineering is a critical step in the development of the system, particularly in the context of keystroke dynamics, where raw data is transformed into a format that enhances the performance of machine learning models. The keystroke dataset consists of timing data collected from the user's keyboard input, such as the time between key presses, the duration of key holds, and the intervals between typing actions.

In this system, manual feature engineering was applied, where various features were extracted from the raw keystroke data to make it suitable for training machine learning models. This process involved calculating key features that might be indicative of a user's unique typing patterns. These features were then normalized and standardized to ensure consistency across data points, allowing the machine learning models to focus on the most relevant information.

Moreover, the system also included a process of feature selection, where only the most influential features were retained based on their predictive power, thereby reducing the dimensionality of the data and improving model efficiency. These features were then used to train and evaluate different machine learning algorithms. The engineered features played a pivotal role in enhancing model accuracy and minimizing the possibility of misclassification, which is critical in authentication systems.

10.2. Model Evaluation:

Model Evaluation was a crucial step in determining the effectiveness of the machine learning models for user authentication. The evaluation process focused on selecting the best-performing models to be implemented in the BAuth system to ensure high accuracy.

To evaluate the models, the initial approach was to use the accuracy score as the primary metric. The dataset was split into training and testing sets using an 80-20 split, where 80% of the data was used for training the models and 20% was used for testing their performance. This method helped assess how well the models could generalize to unseen data, simulating real-world scenarios where the system encounters new user inputs for prediction.

Several machine learning algorithms were implemented and tested during the evaluation phase. The Random Forest model, in particular, emerged as the best performer, achieving an accuracy score of 0.9301 (93.01%). The Support Vector Machine (SVM) followed as the second-best model with an accuracy score of 0.8257

(82.57%). These results demonstrated that the models were able to correctly predict user identity based on their keystroke patterns with a high level of precision. In addition to accuracy, feature selection played a key role in improving the models' performance. By identifying and retaining the most relevant features, the models were able to focus on the most significant data, thereby reducing noise and enhancing prediction reliability. Based on their performance with the best-engineered features, the Random Forest and SVM models were selected for integration into the BAuth system, offering an optimal balance between accuracy and robustness for keystroke-based user authentication.

10.3. API Integration:

API Integration was a vital phase in the implementation of the BAuth system, enabling seamless communication between the machine learning models and the user interface. A RESTful API was developed using Flask, facilitating endpoints for both model training and keystroke classification. During the training process, the API received model parameters and keystroke datasets, triggered training routines, and automatically saved the resulting model files in a dedicated storage directory. For prediction, the API accepted JSON-formatted keystroke data and responded with the predicted user identity, using the currently active model stored on the server. This modular integration allowed the frontend to dynamically trigger backend operations and retrieve results in real time, while abstracting the complexity of model logic from the user interface. Robust error handling, logging, and validation were implemented to ensure reliability and maintainability of the API services.

10.4. Web Interface:

The web interface development focused on creating a user-friendly and functional interface to facilitate administrator interaction with the system. The interface was developed using HTML, CSS, JavaScript, PHP, and integrated with a MySQL backend. Through the interface, the admin user could upload datasets for training, select model types, and view existing model details including performance metrics. The frontend communicated with the backend API via HTTP requests, allowing dynamic content rendering and real-time feedback. Input validation and appropriate messaging mechanisms were included to enhance usability and reduce errors. The interface was designed for clarity and simplicity, ensuring that administrative tasks could be carried out efficiently and securely.

10.5. Deployment:

The deployment of the BAuth system marked the transition from development to a fully operational environment where the system could be tested and demonstrated in real-time. The backend components, including the machine learning models and the Flask API, were hosted locally for controlled testing and evaluation. The trained

models were stored in a dedicated directory within the server environment, allowing the system to dynamically load the active model during authentication simulations. The frontend was connected to the backend through RESTful API endpoints, facilitating smooth interaction between the user interface and core functionalities. Deployment involved configuring the local server, initializing the MySQL database with the required schema, and ensuring proper routing between the API and frontend modules. This localized deployment approach allowed for iterative refinements while preserving system security and performance integrity.

11. TESTING:

Effective testing was essential to ensure the reliability, accuracy, and usability of the BAuth system. The testing strategy adopted involved both unit testing and usability testing, covering both the technical functionality and user-facing elements of the system. Together, these testing processes contributed to a robust and user-friendly system, ensuring that both the backend logic and the interface aligned with the project's goals of reliability, accuracy, and ease of use.

11.1. Unit Testing:

Unit testing was performed throughout the backend components of the system to verify that individual modules and functions behaved as expected. These tests primarily focused on the Flask API endpoints, model training logic, model performance logging, and authentication logic. For instance, the APIHandler methods such as `trainModelAPI()` and `predictUserAPI()` were tested with mock data to confirm their ability to handle different types of input, return the correct predictions, and manage errors gracefully.

The machine learning pipeline was also tested with dummy datasets to confirm the correct loading, preprocessing, training, and serialization of models. Special attention was given to edge cases, such as missing fields in JSON inputs or the absence of an active model, to ensure the system responded predictably and securely. These unit tests were carried out using standard Python testing libraries such as `unittest` and `pytest`, enabling automated and repeatable testing throughout development.

11.2. Usability Testing:

Usability testing was conducted to assess how intuitive and efficient the system interface was for administrative users. As BAuth is intended for use solely by system administrators, the goal was to ensure that navigation, task execution (such as user management, model training, and testing), and feedback mechanisms were clear and responsive.

During this phase, three testers were asked to simulate common tasks such as assigning authentication types, initiating model training, and simulating login attempts. Their feedback was used to refine the user interface—such as improving input validation messages, reorganizing menu flows, and clarifying button labels (e.g., in the test interface where the admin selects the true outcome of a prediction).

The seamless integration between the frontend and the API was also validated to ensure that responses were accurately rendered and that system feedback (success or error) was immediate and meaningful.

12. END-PROJECT REPORT:

12.1. Project Shortcomings:

The BAuth system, specifically using keystroke dynamics as a form of verification, has several inherent limitations that could undermine its effectiveness and usability in real-world applications. These limitations stem from both the nature of keystroke dynamics itself and the design choices made during the development of the system.

1. **Device and Hardware Dependency:**

One of the most significant limitations of using keystroke dynamics is its dependence on the device or keyboard used for typing. The typing pattern of a user can vary greatly depending on factors such as keyboard type, size, and even the responsiveness of the device. A user's typing behavior on one device (e.g., a laptop) might be different on another (e.g., a mobile device or a desktop keyboard). This lack of portability significantly impacts the system's reliability when users switch devices, as it may lead to inaccurate predictions and authentication failures.

2. **Inconsistencies in Typing Behavior:**

Typing behavior is not static, and many internal and external factors can cause significant variations in how a user types. For example, a user may type faster or slower depending on their physical condition, emotional state, or time of day. Users may also alternate between typing with one hand or two hands, which affects typing rhythm. These variations make keystroke dynamics less reliable for consistent, long-term user verification, as the system may fail to recognize a user when their typing pattern deviates from the previously established baseline.

3. **Limited Security Focus:**

The primary focus of the BAuth system is on exploring the potential of keystroke dynamics as a form of user verification, rather than providing comprehensive security features. Security aspects such as encryption, secure API handling, and robust session management are not adequately addressed. As a result, while the system may be effective at distinguishing users based on their typing patterns, it does not offer the level of security required for critical applications where data confidentiality and integrity are paramount.

4. **Lack of Advanced Evaluation Metrics:**

The evaluation of the models used in the BAuth system was based solely on accuracy, which is a limited metric for assessing performance, especially for tasks as nuanced as user authentication. More advanced metrics such as precision, recall, F1-score, and ROC-AUC could provide deeper insights into

model performance and help identify potential areas of improvement, especially in cases where false positives or false negatives carry significant consequences. The failure to explore these metrics leaves potential gaps in understanding the model's full effectiveness.

5. **Non-Standard Authentication for Critical Use:**

Keystroke dynamics, while promising, is still considered a secondary form of authentication in most contexts. Given the relatively high false rejection rates and dependency on consistent typing patterns, relying solely on keystroke dynamics for critical authentication processes may not be practical. This limitation makes the system unsuitable for high-security applications where multi-factor authentication is often required.

Despite these limitations, the project offers valuable insights into keystroke dynamics-based authentication, providing a foundation for future improvements and extensions in the field.

12.2. Project Challenges:

Throughout the development of the BAuth system, several challenges emerged that impacted both the technical execution and the personal aspects of working on the project.

- **Work-Life-Study Balance as an International Student:**

As an international student, balancing work, academic responsibilities, and project development proved to be one of the most significant challenges. The demands of coursework and other academic responsibilities often conflicted with the time needed for the development of the BAuth system. Managing these responsibilities required meticulous time management, and at times, it was difficult to allocate sufficient time to the project. The work-life-study balance was frequently strained, and this led to delays in certain stages of the project, particularly when aligning the project's deadlines with academic obligations.

- **Time Constraints:**

The project was carried out within a set time frame, and as a result, certain aspects had to be rushed or left incomplete. From testing the full range of possible edge cases in the model's prediction accuracy to implementing additional features or conducting deeper security audits, time constraints affected the thoroughness of the system. This also impacted the extent to which usability testing could be carried out, limiting the ability to understand how real-world users would interact with the system.

- **Financial and Emotional Challenges:**

As an international student, managing both the project and financial responsibilities was a significant burden. During the course of the project, I faced an unexpected financial setback when the university's account team suspended my account due to the late payment of my tuition fee deposit. This

suspension resulted in a temporary loss of access to important resources, including university-provided computing tools, libraries, and other academic services.

The delay in the payment caused immense emotional stress, as it created uncertainty about my academic standing and the continuation of my studies. The emotional toll of this situation was substantial, as I had to balance the pressure of completing the project with the anxiety of resolving the financial issue. This hindered my progress in the project.

12.3. Future Enhancements:

Although the BAuth system successfully demonstrated the potential of keystroke-based authentication, there are several areas where the system can be enhanced in the future to improve its functionality, security, and user experience. Some of the key future enhancements include:

1. **Advanced Evaluation Metrics:**

The current evaluation of the models primarily focused on accuracy. However, more advanced evaluation metrics such as precision, recall, F1-score, and ROC-AUC could be incorporated to provide a deeper analysis of model performance. These metrics would help assess the models' ability to detect both true positives and false positives, improving the robustness of the system, especially in scenarios with imbalanced data.

2. **Security Enhancements:**

Although the current project prioritizes the exploration of keystroke authentication, the core security features were not deeply implemented. Future versions of the system should consider integrating encryption for keystroke data transmission, and applying advanced anomaly detection techniques to further enhance security and mitigate risks.

3. **Cross-Platform Compatibility:**

As user typing patterns are highly influenced by the device and keyboard used, ensuring that the system can adapt to various devices (e.g., laptops, mobile devices, virtual keyboards) is essential. Future versions of the system could explore solutions to make keystroke authentication more platform-agnostic and ensure a seamless user experience across different devices.

4. **Incorporating Other Biometric Data:**

To improve the reliability and accuracy of user authentication, combining keystroke dynamics with other biometric data, such as mouse movement could be a potential enhancement. This multi-modal biometric authentication would add an extra layer of security while maintaining the convenience of using keystroke patterns.

5. **Scalability:**

For large organizations, the ability to scale the BAuth system to handle thousands of users would be crucial. Future versions of the system could improve scalability by implementing more efficient data storage solutions,

better handling of model training, and distributed computing to speed up model training and predictions.

12.4. General Evaluation:

The BAuth project culminated in the successful development of a keystroke-based user authentication system tailored for administrative use. The project followed Agile methodology, progressing through iterative sprints that enabled continuous refinement and timely delivery of core functionalities. From initial research and risk assessment through feature engineering, model training, evaluation, and system integration, each phase contributed to building a robust and secure authentication platform. The final system featured a functional web interface, integrated machine learning models with high accuracy (notably Random Forest with 93.01%), and a streamlined API for real-time prediction. Testing ensured both technical reliability and administrative usability. Overall, the project met its objectives, delivering a novel and practical tool for keystroke authentication.

13. REFLECTIONS:

This project has been a challenging yet rewarding journey, providing valuable insights into both the technical and personal aspects of the project.

From a technical perspective, the biggest learning experience came from the integration of machine learning into a practical authentication system. Understanding the intricacies of feature engineering and selecting the right model were crucial in achieving high performance.

On a personal level, this project has been an incredible opportunity for growth. Not only did I enhance my technical skills in machine learning, API development, and web frontend development, but I also learned valuable lessons in project management and resilience. Managing the scope of the project, adhering to deadlines, and navigating through obstacles helped me develop better problem-solving and time management skills.

The development of the BAuth system has been a challenging but fulfilling experience. There were hurdles during the project but it also served as a reminder of the importance of resilience and maintaining a steady focus on long-term goals despite short-term setbacks. While the project is complete, I am excited about the potential for future enhancements and the opportunity to further explore the evolving field of biometric-based authentication.

14. CONCLUSION:

The BAuth project has successfully achieved its primary goal of developing a keystroke-based authentication system, utilizing machine learning to authenticate users based on their typing patterns. This project demonstrated the power of machine learning techniques to provide an innovative approach to user verification.

Through careful consideration of design, implementation, and testing, the system was able to provide a reliable and scalable solution for secure user authentication. The project also highlighted the importance of careful planning, resource management, and adherence to project timelines, all while considering the personal challenges that arose during its development. By adhering to Agile principles and sprint planning, the project was able to evolve iteratively, incorporating feedback and adjustments along the way.

While the project achieved its intended objectives, it also revealed certain limitations, such as the variability of user typing patterns and hardware dependencies, which could affect the system's overall accuracy. Additionally, security considerations were secondary in this project, as the system was designed to integrate into existing user management infrastructures.

In conclusion, the BAuth system serves as a foundational step toward enhancing authentication systems using behavioral biometrics. While there are areas for improvement, the project has laid the groundwork for future advancements in biometric-based user authentication. It also provided valuable hands-on experience, contributing to personal and professional growth, and underscoring the importance of resilience and problem-solving in project development.

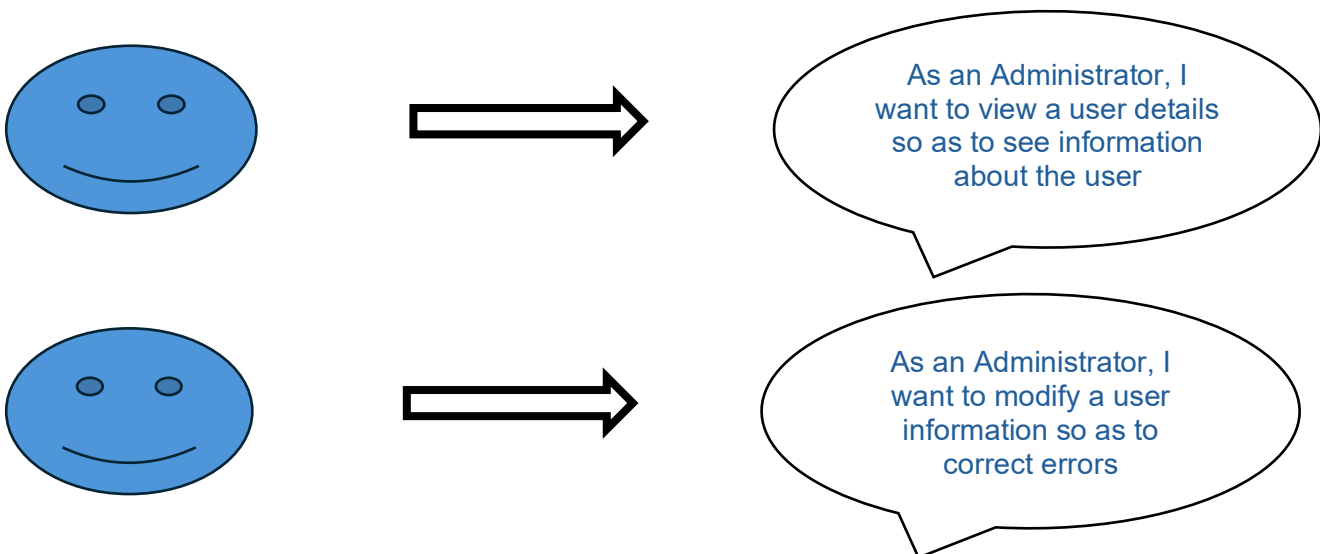
15. REFERENCE LIST:

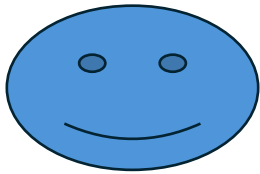
1. Gaines, R., Lisowski, W., Press, S.J. and Shapiro, N. (1980) *Authentication by keystroke timing: Some preliminary results*. Santa Monica, CA: RAND Corporation.
2. AbdelRaouf, H., Chelloug, S.A., Muthanna, A., Semary, N., Amin, K. and Ibrahim, M. (2023) 'Efficient convolutional neural network-based keystroke dynamics for boosting user authentication', *Sensors*, 23(10), p.4898. <https://doi.org/10.3390/s23104898>
3. Wang, X. and Hou, D. (2024) 'Enhancing keystroke dynamics authentication with ensemble learning and data resampling techniques', *Electronics*, 13(22), p.4559. <https://doi.org/10.3390/electronics13224559>
4. Wang, X., Shi, Y., Zheng, K., Zhang, Y., Hong, W. and Cao, S. (2022) 'User authentication method based on keystroke dynamics and mouse dynamics with scene-irrelevant features in hybrid scenes', *Sensors*, 22(17), p.6627. <https://doi.org/10.3390/s22176627>
5. Wang, X., Zheng, Q., Zheng, K. and Wu, T. (2020) 'User authentication method based on MKL for keystroke and mouse behavioral feature fusion', *Security and Communication Networks*, 2020, Article ID 9282380, 14 pages. <https://doi.org/10.1155/2020/9282380>
6. Elliot, K., Graham, J., Yassin, Y., Ward, T., Caldwell, J. and Attie, T. (2019) 'A comparison of machine learning algorithms in keystroke dynamics', *Proceedings of the 2019 International Conference on Computational Science*

- and Computational Intelligence (CSCI), Las Vegas, NV, USA, 5–7 December, pp. 127–132. <https://doi.org/10.1109/CSCI49370.2019.00028>
7. AlZain, M.A. and Pardede, E. (2013) ‘Security aspects of biometric authentication systems: A literature review’, *International Journal of Computer Applications*, 61(10), pp. 1–6.
 8. Burrell, J. (2016) ‘How the machine “thinks”: Understanding opacity in machine learning algorithms’, *Big Data & Society*, 3(1), pp. 1–12. <https://doi.org/10.1177/2053951715622512>
 9. EU (2016) *General Data Protection Regulation (GDPR)*. [online] Available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> [Accessed 18 Apr. 2025].
 10. Floridi, L., Cowls, J., Beltrametti, M. et al. (2018) ‘AI4People—An ethical framework for a good AI society: Opportunities, risks, principles, and recommendations’, *Minds and Machines*, 28(4), pp. 689–707. <https://doi.org/10.1007/s11023-018-9482-5>
 11. Jain, A.K., Ross, A. and Nandakumar, K. (2016) *Introduction to biometrics*. 1st ed. Springer.
 12. Van der Ploeg, I. (2005) ‘The machine-readable body: Essays on biometrics and the informatisation of the body’, *Maastricht: Shaker Publishing*.
 13. Tudorache, B. (2023) ‘The application of ML models in user recognition via keystroke dynamics’, *HackerNoon*, 10 October. Available at: <https://hackernoon.com/the-application-of-ml-models-in-user-recognition-via-keystroke-dynamics> (Accessed: 11 April 2025).

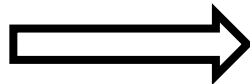
16. APPENDICES:

16.1. User Stories:

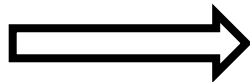




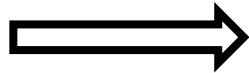
As an Administrator, I want to delete a user so as to remove the user from the system



As an Administrator, I want to train a model so as to have a machine learning model for the system performance



As an Administrator, I want to view a model details so as to see information about the model, including the model performance



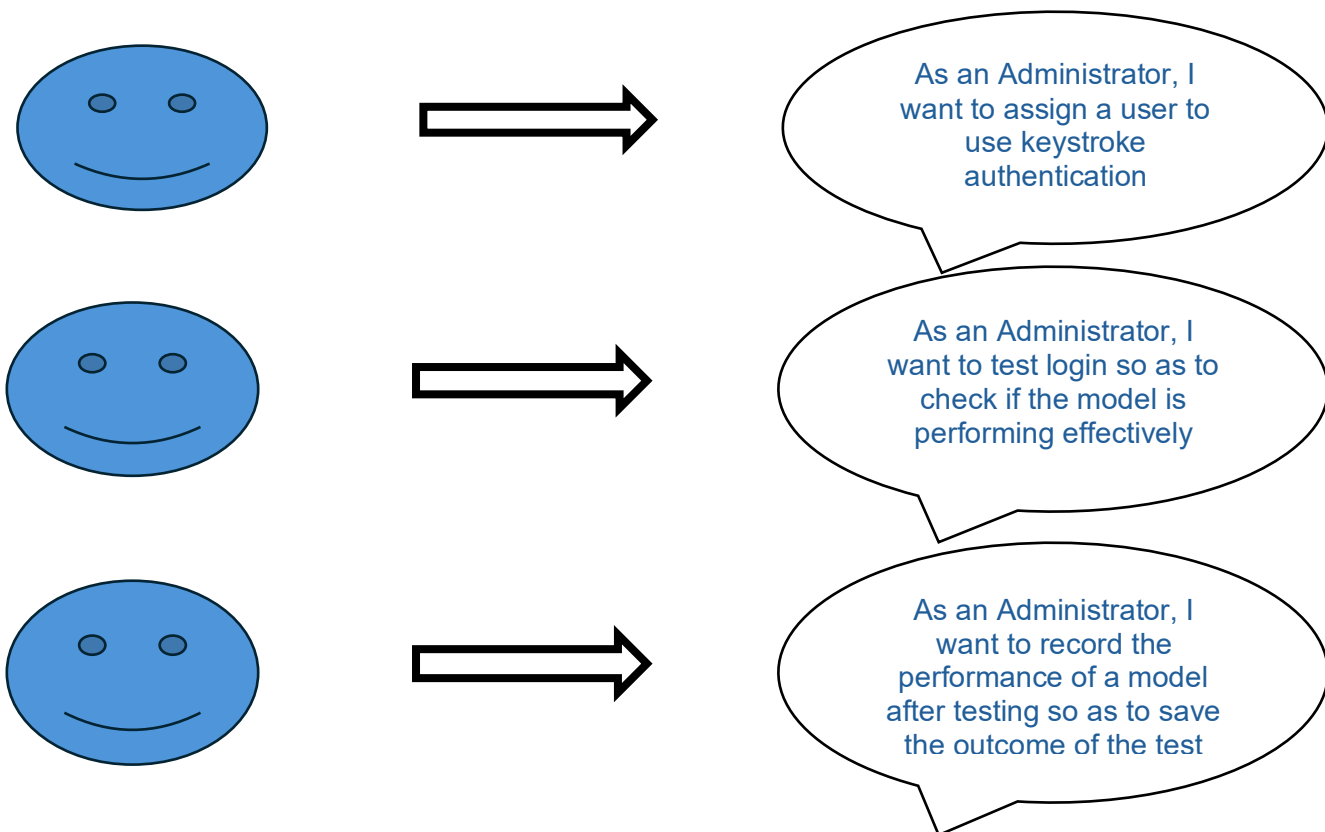
As an administrator, I want to edit a model name so as to change the name of the model



As an administrator, I want to delete a model so as to remove the model from the system



As an Administrator, I want to activate a model so as to run authentication functions with the model



16.2. Wireframes:

The full responsive wireframe of the system can be accessed at

<https://www.figma.com/proto/BoY4rPrQYqwYkuCYH0724R/BAuth-wireframe?node-id=2-5&p=f&t=WUtTPlctnhmWDCDe-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=2%3A5>

Below are snippet of the designs:

Users

Users	Model	Authentication	Test																				
<table> <thead> <tr> <th>User ID</th> <th>Username</th> <th>Password</th> <th>Auth Type</th> </tr> </thead> <tbody> <tr> <td>S002</td> <td>simpsons</td> <td>#0vnhdokw&1Fh</td> <td>Password + ks</td> </tr> <tr> <td>S003</td> <td>kelvin</td> <td>#0vnhdokw&1Fh</td> <td>#0vnhdokw&1Fh</td> </tr> <tr> <td>S004</td> <td>uchenna</td> <td>#0vnhdokw&1Fh</td> <td>#0vnhdokw&1Fh</td> </tr> <tr> <td>S005</td> <td>amokwu</td> <td>#0vnhdokw&1Fh</td> <td>#0vnhdokw&1Fh</td> </tr> </tbody> </table>	User ID	Username	Password	Auth Type	S002	simpsons	#0vnhdokw&1Fh	Password + ks	S003	kelvin	#0vnhdokw&1Fh	#0vnhdokw&1Fh	S004	uchenna	#0vnhdokw&1Fh	#0vnhdokw&1Fh	S005	amokwu	#0vnhdokw&1Fh	#0vnhdokw&1Fh			
User ID	Username	Password	Auth Type																				
S002	simpsons	#0vnhdokw&1Fh	Password + ks																				
S003	kelvin	#0vnhdokw&1Fh	#0vnhdokw&1Fh																				
S004	uchenna	#0vnhdokw&1Fh	#0vnhdokw&1Fh																				
S005	amokwu	#0vnhdokw&1Fh	#0vnhdokw&1Fh																				

View User

Users	Model	Authentication	Test								
<table> <thead> <tr> <th>User ID</th> <td>S005</td> </tr> </thead> <tbody> <tr> <td>Username</td> <td>amokwu</td> </tr> <tr> <td>Password</td> <td>#0vnhdokw&1Fh</td> </tr> <tr> <td>Auth Type</td> <td>Password + keystroke</td> </tr> </tbody> </table> Assign Auth Edit DeleteUser	User ID	S005	Username	amokwu	Password	#0vnhdokw&1Fh	Auth Type	Password + keystroke			
User ID	S005										
Username	amokwu										
Password	#0vnhdokw&1Fh										
Auth Type	Password + keystroke										

Train Model

Users

Model

Authentication

Test

Model Name :

Select dataset :

File (CVS Only)

Model Type :

SELECT

V

Train Model

Clear All

Models

Users

Model

Authentication

Test

Model ID	Name	Type	Active
M001	Model001	Random Forest	☒
M003	ModelTest	SVM	☐
M004	ModInitial	SVM	☐
M005	Model0	Random Forest	☐

View Model

Users

Model

Authentication

Test

Model ID	M001
Model Name	Model001.py
Dataset	Keystroke_dynamics
Model Type	Random Forest
Path/Location	User/sally/BAuth/model
Date Trained	2025-03-24 10:30:00

View Performance

Edit

Delete Model

View Model Performance



Auth

Users

Model

Authentication

Test

Use the buttons to assign users to secondary authentication

Search

X

User Id	Username
S002	simpsons
S003	kelvin
S004	uchenna
S005	amokwu

Keystroke

S003

S004

S005

SAVE

Auth Success

Users

Model

Authentication

Test

SUCCESSFUL !

OK

Test

Users

Model

Authentication

Test

Keystroke reflect the keystroke of user typing the string ".tie5Roani"

Username

Password

Keystroke (JSON)

Login

Reset

Login

Users

Model

Authentication

Test

MODEL OUTCOME !

Username : amokwu

Predicted User : amokwu

please select model performance below

True Positive

True Negative

False Positive

False Negative

16.3. About Dataset:

Benchmark Dataset *From Kaggle*

<https://www.kaggle.com/datasets/carnegiecylab/keystroke-dynamics-benchmark-data-set/data>

Description: The data consist of keystroke-timing information from 51 subjects (typists), each typing a password (**.tie5Roanl**) 400 times.

The data are arranged as a table with 34 columns. Each row of data corresponds to the timing information for a single repetition of the password by a single subject. The first column, subject, is a unique identifier for each subject (e.g., s002 or s057). Even though the data set contains 51 subjects, the identifiers do not range from s001 to s051; subjects have been assigned unique IDs across a range of keystroke experiments, and not every subject participated in every experiment. For instance, Subject 1 did not perform the password typing task and so s001 does not appear in the data set. The second column, sessionIndex, is the session in which the password was typed (ranging from 1 to 8). The third column, rep, is the repetition of the password within the session (ranging from 1 to 50).

The remaining 31 columns present the timing information for the password. The name of the column encodes the type of timing information. Column names of the form H.key designate a hold time for the named key (i.e., the time from when key was pressed to when it was released). Column names of the form DD.key1.key2 designate a keydown-keydown time for the named digraph (i.e., the time from when key1 was pressed to when key2 was pressed). Column names of the form UD.key1.key2 designate a keyup-keydown time for the named digraph (i.e., the time from when key1 was released to when key2 was pressed). Note that UD times can be negative, and that H times and UD times add up to DD times.

Consider the following one-line example of what you will see in the data:

```
subject sessionIndex rep H.period DD.period.t UD.period.t ...  
s002 1 1 0.1491 0.3979 0.2488 ...
```

The example presents typing data for subject 2, session 1, repetition 1. The period key was held down for 0.1491 seconds (149.1 milliseconds); the time between pressing the period key and the t key (keydown-keydown time) was 0.3979 seconds; the time between releasing the period and pressing the t key (keyup-keydown time) was 0.2488 seconds; and so on.

For BAuth system, the training and testing datasets on the gitHub repo are a reflection of the above dataset from Kaggle. The data was shuffled and spitted into two datasets (train_data and test_data). The train_data meant to be used in training the models, and the records from the test_data to be used for login testing.

16.4. API Endpoints:

Flask API runs locally on port 5000 with the url <http://127.0.0.1:5000>.

There are two API endpoints for training a model and running a prediction.

- To train a model, make a POST request to <http://127.0.0.1:5000/train> with the following data:

`dataset` - a cvs file.

`model\_name` - string indicating the model name.

`model\_type` - string of either 'RF' or 'SVM'.

The API trains a machine learning model and saves the resulting model files (model_name.joblib, and model_name_scaler.joblib) to the local directory of the PC ([home_dir]/BAuth/model_files/)

- To run a prediction, make a POST request to <http://127.0.0.1:5000/predict> with a JSON data of the following format. Note the required keys:

```
{  
  "model_name": [name of the active model],  
  "H.period": [float value],  
  "DD.period.t": [float value],  
  "UD.period.t": [float value],  
  "H.t": [float value],  
  "DD.t.i": [float value],  
  "UD.t.i": [float value],  
  "H.i": [float value],  
  "DD.i.e": [float value],  
  "UD.i.e": [float value],  
  "H.e": [float value],  
  "DD.e.five": [float value],  
  "UD.e.five": [float value],  
  "H.five": [float value],  
  "DD.five.Shift.r": [float value],  
  "UD.five.Shift.r": [float value],  
  "H.Shift.r": [float value],  
  "DD.Shift.r.o": [float value],  
  "UD.Shift.r.o": [float value],  
  "H.o": [float value],  
  "DD.o.a": [float value],  
  "UD.o.a": [float value],  
  "H.a": [float value],  
  "DD.a.n": [float value],  
  "UD.a.n": [float value],  
  "H.n": [float value],  
  "DD.n.l": [float value],  
  "UD.n.l": [float value],  
  "H.l": [float value],
```

```

    "DD.I.Return": [float value],
    "UD.I.Return": [float value],
    "H.Return": [float value]
}

```

For effective model training, the cvs dataset must be formatted to include the following column headers:

```

[subject, H.period, DD.period.t, UD.period.t, H.t, DD.t.i, UD.t.i, H.i, DD.i.e,
UD.i.e, H.e, DD.e.five, UD.e.five, H.five, DD.five.Shift.r, UD.five.Shift.r,
H.Shift.r, DD.Shift.r.o, UD.Shift.r.o, H.o, DD.o.a, UD.o.a, H.a, DD.a.n, UD.a.n,
H.n, DD.n.I, UD.n.I, H.I, DD.I.Return, UD.I.Return, H.Return].

```

This is a reflection of the dataset description in Appendix 16.3 above.

16.5. Running the System:

Setup Instructions:

Python, PHP, and MySQL are required for the system to run. Make sure they are installed and configured correctly.

- Download and install Python from [python official website](https://www.python.org/).

PHP and MySQL comes together with XAMPP, a free and open-source cross-platform web server solution.

- [Download](#) and install XAMPP.
- Copy BAuth directory from github repo to XAMPP's htdocs directory on your local machine.

Moving forward, required python dependencies are listed in requirements.txt file on the github repo. To install, run command `pip install -r requirements.txt` on your device terminal.

Running the system:

Flask is the python library that provides the API for the model training and classification. To run Flask, navigate to the directory that contains the python files (Python_scripts from github repo) in terminal or command line. Then run the command `python bauth\_app.py` or `python3 bauth\_app.py` to execute.

[Output: Running on <http://127.0.0.1:5000/>]

With Flask running,

- Start XAMPP and run servers.
- Open <http://localhost/phpmyadmin/> on your browser, and create a new database with the name "bauth". Then import keystroke.sql (from github repo) into the newly created database to populate the database with required tables and data.
- Open <http://localhost/bauth/> on your browser to access the BAuth system.

API

The API run independently of the web interface. With Flask running, you can make a POST api call request to <http://127.0.0.1:5000/train> to train a model, supplying inputs such as - model_name(string), model_type('RF' or 'SVM'), and dataset(csv file). The process saves model files (model.joblib and scaler.joblib) to your local machine at [home]/BAuth/model_files/.

Note that the dataset must be formatted appropriately for effective model training.

The dataset definition and format is contained in section 16.4. of this report.

[Use already formatted Dataset/train_data.csv from the github repo to train models]

To make a prediction, make a POST request to <http://127.0.0.1:5000/predict> , supplying a JSON object containing model name and keystroke data. The JSON object definition and format is contained in section 16.4. of this report.

[For convenience, some sample test data are contained in sample_test_json.docx]

The prediction returns user ID of the predicted user.

16.6. Product Manual:

Navigating the System

- The home page shows shortcuts to sections like Users, Model, Authentication, and Test.

Users

- **View Users:** Go to **Users > View** to see registered users.
- **Edit a User:** Click **Edit** to update username, password, or auth type.
- **Delete a User:** Click **Delete** to remove a user.
- **Assign User to Keystroke Auth:** Go to **Users > Assign Auth**. To remove, go to **Users > Edit**.

Model

- **Train a Model:** Go to **Model > Train**, enter a model name, select a dataset (CSV), choose a model type (SVM or Random Forest), and click **Train**.
- **View Models:** View all trained models via **Model > View**.
- **Activate a Models:** Activate a model to be used for authentication testing via **Model > View > Activate**. Note that only one model can be active at a time, activating a model deactivates the previous active model.
- **Model Performance:** See model prediction accuracy charts at **Model > Performance**.

Authentication

- **List by Auth Type:** View users categorized by authentication type via **Authentication > List by Auth Type**.
- **Assign Keystroke Authentication:** Go to **Authentication > Assign** to enable keystroke authentication for users.

Testing Authentication

- Visit **Test**.
- Enter username, password, and paste the keystroke JSON data.

- Click **Test to Predict API**.
- **Record prediction results**: True Positive, True Negative, False Positive, False Negative.

Behind the Scenes

- Trained models are automatically saved in your system's local directory.